Practical 01

Here's a **detailed explanation** of Python sequence data types and related concepts, covering all your questions thoroughly.

---

1. What are sequence data types in Python?

Sequence data types are **collections of items arranged in an ordered manner**, where each element has a **position (index)**.

Common Sequence Data Types

- **Strings (`str`)**
- **Lists (`list`)**
- **Tuples (`tuple`)**
- **Range (`range`)**

(Although **arrays, dictionaries, and sets** are often discussed alongside them, strictly speaking only the above are true *sequence types* because they are ordered and indexable.)

Indexing in Sequences

Indexing allows you to access elements using position.

- Starts from **0** (left to right)
- Negative indexing starts from **-1** (right to left)

```
s = "Python"
print(s[0])   # P
print(s[-1])  # n
```

Slicing in Sequences

Slicing extracts a portion of the sequence.

```
sequence[start:stop:step]
```

Example:

```
s = "Python"
print(s[0:4])   # Pyth
print(s[::2])   # Pto
```

Works similarly for:

- Lists
- Tuples
- Strings
- Range (converted if needed)

---

2. Difference Between List and Tuple

Feature	List	Tuple
Mutability	Mutable	Immutable
Syntax	`[ ]`	`( )`
Performance	Slower	Faster
Memory	More	Less

Example

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)

my_list[0] = 10   # Allowed
# my_tuple[0] = 10 → Error
```

Why prefer Tuple over List?

1. **Data safety**

Tuples cannot be changed accidentally.

2. **Faster execution**

Tuples are optimized for fixed data.

3. **Used as dictionary keys**

Because they are immutable.

4. **Real-world usage**

- Coordinates: `(x, y)`
- Database records

- Function return values

---

3. Arrays vs Lists in Python

Python provides arrays via the `array` **module**.

Key Differences

Feature	List	Array
Data type	Mixed allowed	Same type only
Flexibility	High	Limited
Memory	More	Less
Speed	Slower	Faster (numeric data)

Example

```
import array

arr = array.array('i', [1, 2, 3])
```

When to use Arrays?

Use arrays when:

- Working with **large numeric data**
- Need **memory efficiency**
- Need **faster numerical operations**

Otherwise, lists are preferred.

---

4. How Dictionaries Work Internally

Dictionaries are based on a **hash table**.

Key Concepts

- Each key is passed through a **hash function**
- The hash determines where data is stored in memory

```
d = {"a": 1, "b": 2}
print(d["a"])  # 1
```

Why Dictionary Lookups Are Fast

- Time complexity: **O(1)** (average case)
- Direct access using hash values
- No need to scan entire data

Comparison with List/Tuple

- List lookup: **O(n)** (linear search)
- Dictionary lookup: **O(1)**

Important Notes

- Keys must be **immutable**
- Collisions handled internally

---

5. Sets vs Lists

Differences

Feature	List	Set
Order	Ordered	Unordered
Duplicates	Allowed	Not allowed
Indexing	Yes	No
Performance	Slower for search	Faster for membership

Example

```
my_list = [1, 2, 2, 3]
my_set = {1, 2, 2, 3}

print(my_set)  # {1, 2, 3}
```

How Sets Handle Duplicates

- Automatically remove duplicates
- Store only **unique elements**

Set Operations

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a | b)  # Union
```

```
print(a & b)  # Intersection
print(a - b)  # Difference
```

Practical Uses of Sets

- Removing duplicates
- Fast membership testing
- Mathematical operations (union, intersection)
- Finding common elements

Practicale uses

String Operation

```
s = "Python Programming"

print("Original String:", s)

# Indexing
print("First character:", s[0])

# Slicing
print("Slice (0:6):", s[0:6])

# Length
print("Length:", len(s))

# Upper & Lower
print("Upper:", s.upper())
print("Lower:", s.lower())

# Replace
print("Replace:", s.replace("Python", "Java"))

# Concatenation
print("Concat:", s + " Language")

# Membership
print("Contains 'Python'?", "Python" in s)
```

List Operation

```
# List operations
lst = [10, 20, 30, 40]
```

```
print("Original List:", lst)

# Append
lst.append(50)
print("After append:", lst)

# Insert
lst.insert(1, 15)
print("After insert:", lst)

# Remove
lst.remove(30)
print("After remove:", lst)

# Pop
lst.pop()
print("After pop:", lst)

# Slicing
print("Slice:", lst[1:3])

# Sort
lst.sort()
print("Sorted:", lst)

# Reverse
lst.reverse()
print("Reversed:", lst)
```

Array Operations

```
import array

# Create array
arr = array.array('i', [1, 2, 3, 4])

print("Original Array:", arr)

# Append
arr.append(5)
print("After append:", arr)

# Insert
arr.insert(1, 10)
print("After insert:", arr)

# Remove
arr.remove(3)
print("After remove:", arr)

# Indexing
print("Element at index 2:", arr[2])
```

```
# Slicing
print("Slice:", arr[1:4])
```

Tuple Operation

```
# Tuple operations
t = (1, 2, 3, 4, 2)

print("Original Tuple:", t)

# Indexing
print("First element:", t[0])

# Slicing
print("Slice:", t[1:4])

# Count
print("Count of 2:", t.count(2))

# Index
print("Index of 3:", t.index(3))

# Length
print("Length:", len(t))
```

Dictionary Operation

```
# Dictionary operations
d = {"name": "Alice", "age": 21}

print("Original Dictionary:", d)

# Access
print("Name:", d["name"])

# Add/Update
d["age"] = 22
d["city"] = "Delhi"
print("After update:", d)

# Remove
d.pop("age")
print("After pop:", d)

# Keys, Values, Items
print("Keys:", d.keys())
print("Values:", d.values())
print("Items:", d.items())
```

set Operations